Module 2: Type System & Basic Types

1. Introduction to Type System in TypeScript

TypeScript uses a static type system to define variable types, helping prevent errors at compile time.

2. Primitive Types in TypeScript

String

```typescript
let name: string = "Alice";
console.log(name);
```

Number

```typescript
let age: number = 25;
console.log(age);
```

Boolean

```typescript
let isStudent: boolean = true;
console.log(isStudent);
```

3. Special Types in TypeScript

Any

Allows any value type, but removes type safety.

```typescript
let random: any = "Hello";
random = 42;  // No error
```

Unknown

Safer alternative to `any`, requires type checking.

```typescript
```

```typescript
let data: unknown = "Hello";
if (typeof data === "string") {
console.log(data.toUpperCase());
}
```

Void

Used for functions that do not return a value.

```typescript
function logMessage(): void {
console.log("This function returns nothing.");
}
```

Null & Undefined

```typescript
let empty: null = null;
let notDefined: undefined = undefined;
```

4. Type Assertions

Used to override TypeScript's type inference.

```typescript
let value: any = "Hello";
let strLength: number = (value as string).length;
console.log(strLength);
```

5. Type Inference

TypeScript automatically detects variable types.

```typescript
let message = "Hello"; // Inferred as string
let count = 10; // Inferred as number
```

6. Working with Enums

Enums allow defining a set of named constants.

```typescript
enum Direction {
Up,
Down,
Left,
Right
}
let move: Direction = Direction.Up;
console.log(move); // Output: 0
```

Practice Questions:

1. What is the difference between `any` and `unknown` types?
2. Convert the following JavaScript code to TypeScript:

```javascript
let username = "John";
let userAge = 30;
console.log(username.toUpperCase());
```

3. Define an enum for `DaysOfWeek`.
4. What will be the output of this TypeScript code?

```typescript
let value: any = "TypeScript";
console.log((value as string).length);
```

5. What type should be used for a function that does not return a value?

Practice Answers:

1. `any` allows assigning any type, bypassing type checking, while `unknown` requires type checking before use.
2. Converted TypeScript code:

```typescript
let username: string = "John";
let userAge: number = 30;
```

console.log(username.toUpperCase());

3. Enum for `DaysOfWeek`:

typescript

```
enum DaysOfWeek {
Sunday,
Monday,
Tuesday,
Wednesday,
Thursday,
Friday,
Saturday
}
```

4. The output will be `10`, as `"TypeScript"` has 10 characters.

5. Use the `void` type for functions that do not return a value.